Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	**TE**	**Semester:**	**VI**
Course Code:	**CSL601**	**Course Name:**	**Machine Learning Lab**

Name of Student:	**Shravani Sandeep Raut**
Roll No. :	**40**
Experiment No.:	**08**
Title of the Experiment:	**Implementation of Single Layer Perceptron Learning Algorithm**
Date of Performance:	**12/03/2026**
Date of Submission:	**09/04/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Mrs. Sneha Yadav

Signature :

Date:

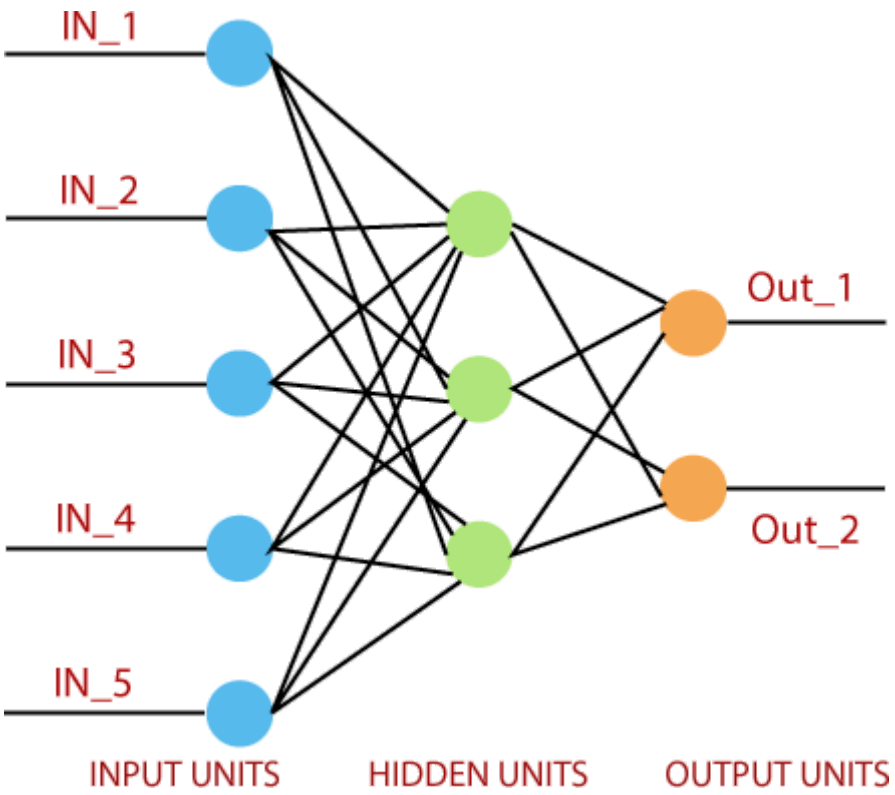
Aim: Implementation of Single Layer Perceptron Learning Algorithm

Objective: Able to implement and understand the aspects of Single Layer Perceptron Learning Algorithm.

Theory:

The perceptron is a single processing unit of any neural network. **Frank Rosenblatt** first proposed in **1958** is a simple neuron which is used to classify its input into one or two categories. Perceptron is a linear classifier, and is used in supervised learning. It helps to organize the given input data.

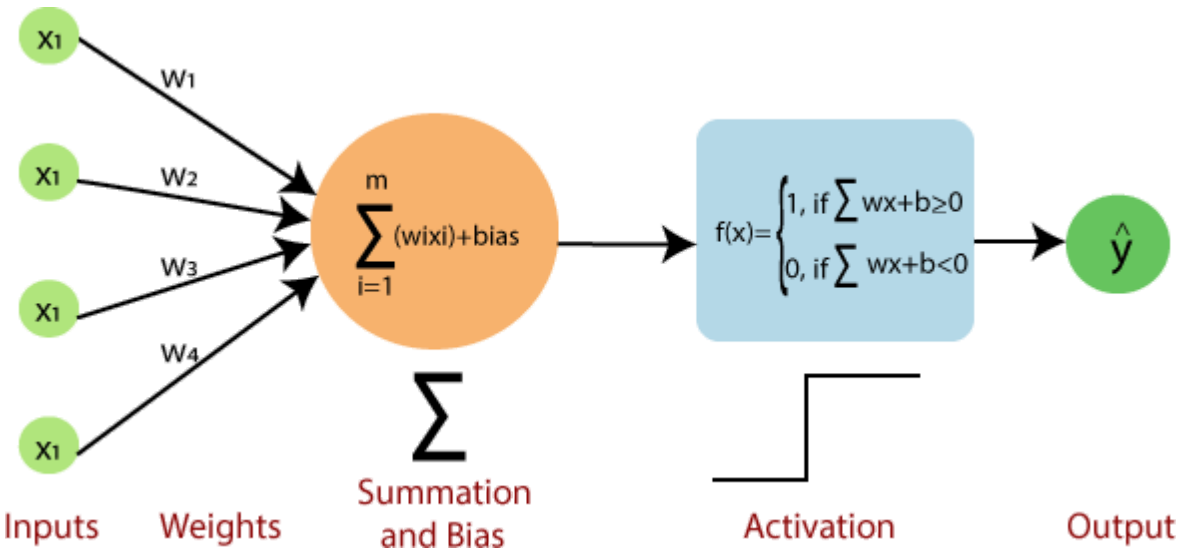
A perceptron is a neural network unit that does a precise computation to detect features in the input data. Perceptron is mainly used to classify the data into two parts. Therefore, it is also known as **Linear Binary Classifier**.



Perceptron uses the step function that returns +1 if the weighted sum of its input 0 and -1.

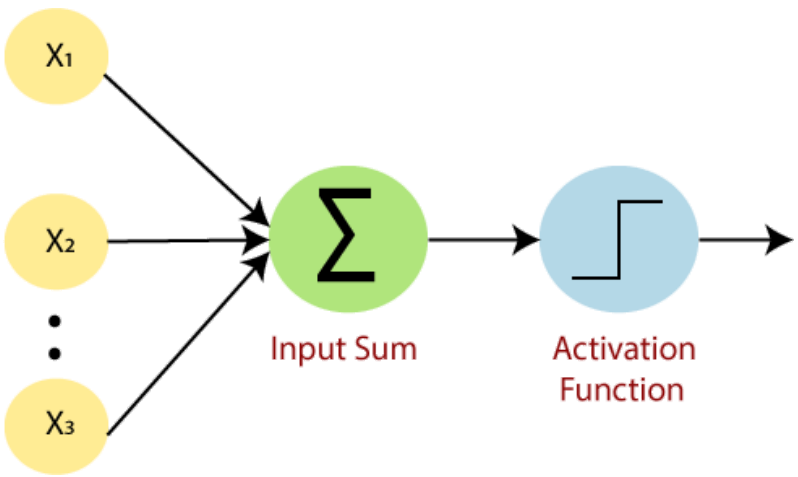
The activation function is used to map the input between the required value like (0, 1) or (-1, 1).

A regular neural network looks like this:

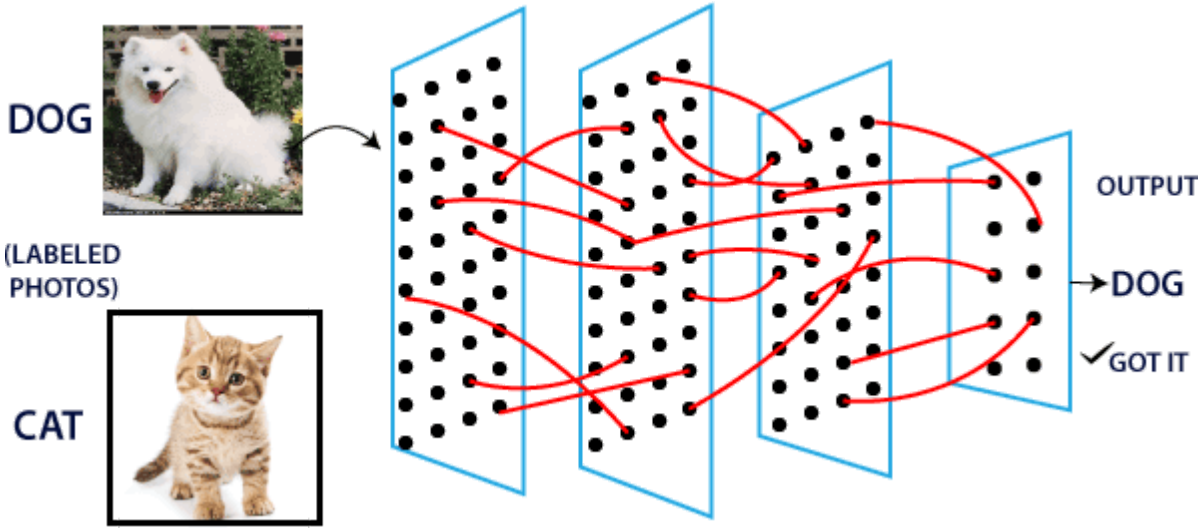


The perceptron consists of 4 parts.

- **Input value or One input layer:** The input layer of the perceptron is made of artificial input neurons and takes the initial data into the system for further processing.
- **Weights and Bias:**
  Weight: It represents the dimension or strength of the connection between units. If the weight to node 1 to node 2 has a higher quantity, then neuron 1 has a more considerable influence on the neuron.
  Bias: It is the same as the intercept added in a linear equation. It is an additional parameter which task is to modify the output along with the weighted sum of the input to the other neuron.
- **Net sum:** It calculates the total sum.
- **Activation Function:** A neuron can be activated or not, is determined by an activation function. The activation function calculates a weighted sum and further adding bias with it to give the result.



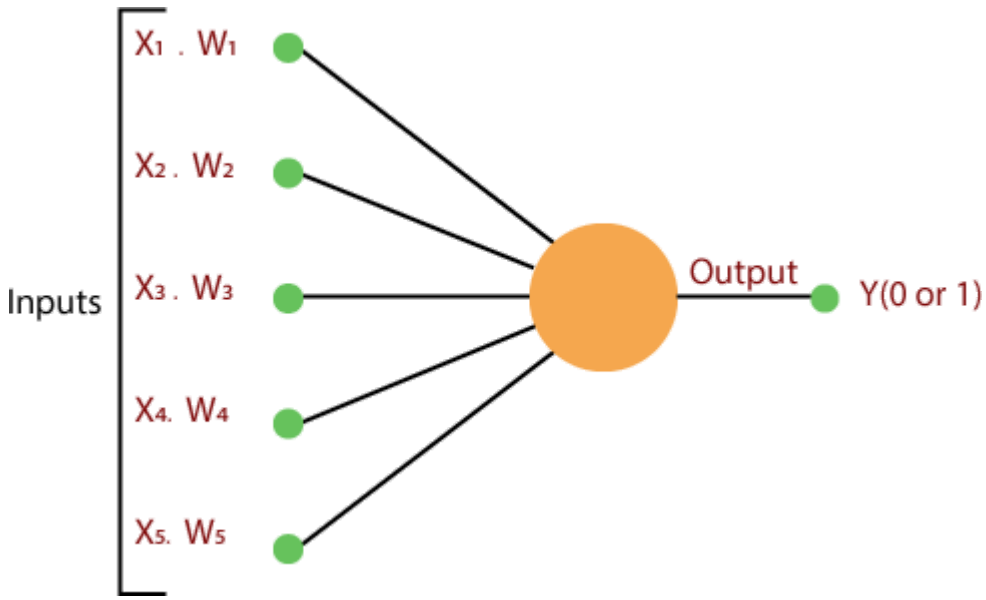
A standard neural network looks like the below diagram.



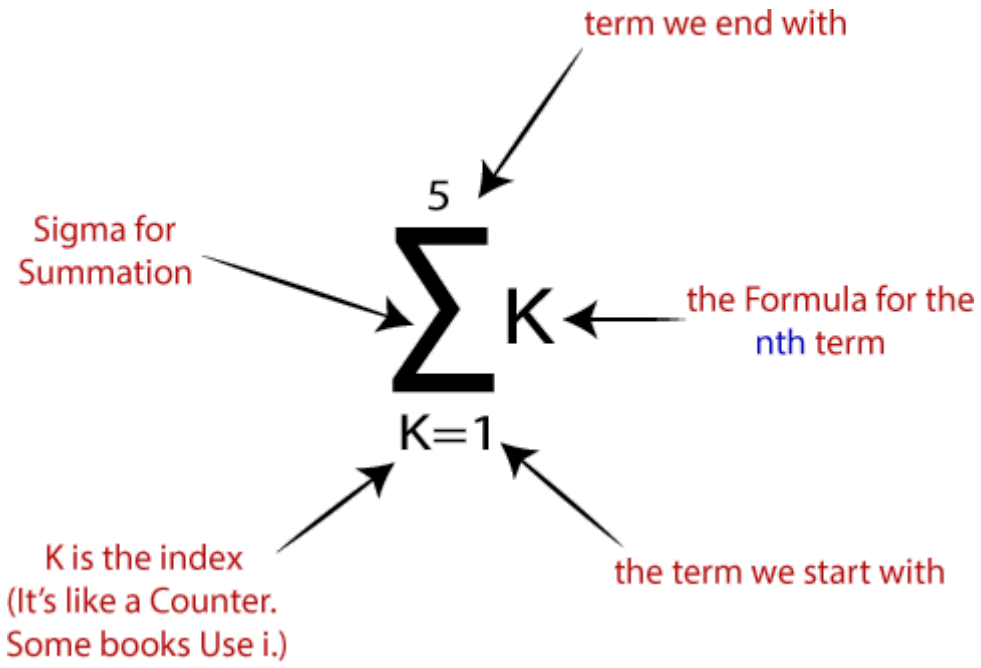
How does it work?

The perceptron works on these simple steps which are given below:

a. In the first step, all the inputs x are multiplied with their weights **w**.

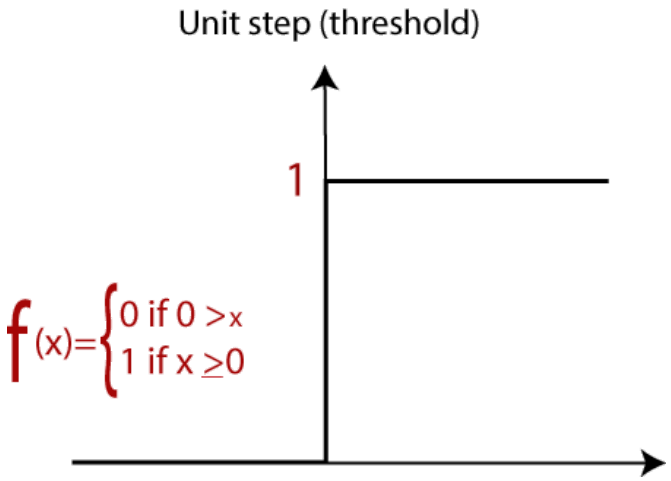


b. In this step, add all the increased values and call them the **Weighted sum**.



c. In our last step, apply the weighted sum to a correct **Activation Function**. **For Example:**

A Unit Step Activation Function



Implementation:

```
import numpy as np

class Perceptron:
    def __init__(self):
        self.w = None
        self.b = 0

    def fit(self, X, y, lr=0.1, epochs=10):
        self.w = np.zeros(X.shape[1])
        for _ in range(epochs):
            for xi, target in zip(X, y):
                y_pred = 1 if np.dot(xi, self.w) + self.b >= 0 else 0
                error = target - y_pred
                self.w += lr * error * xi
                self.b += lr * error

    def predict(self, x):
        return 1 if np.dot(x, self.w) + self.b >= 0 else 0


X = np.array([[0,0],[0,1],[1,0],[1,1]])

# AND Gate
y_and = np.array([0,0,0,1])
p1 = Perceptron()
p1.fit(X, y_and)

print("AND Gate:")
for x in X:
    print(x, "->", p1.predict(x))

# OR Gate
y_or = np.array([0,1,1,1])
p2 = Perceptron()
p2.fit(X, y_or)

print("\nOR Gate:")
for x in X:
    print(x, "->", p2.predict(x))
```

Conclusion:

1. **Comment on understanding of Single Layer Perceptron Learning Algorithm and its use.**

   The Single Layer Perceptron (SLP) Learning Algorithm is a supervised learning method used for binary classification of linearly separable data. It works by computing the weighted sum of inputs, applying a step activation function, and updating weights and bias based on the error between actual and predicted output. It is mainly used for simple classification tasks such as logic gates (AND, OR) and forms the foundation for understanding more advanced neural networks in Artificial Neural Networks (ANN).